Project - High Level Design

on

Construction Content Generator

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr No	Student Name	Enrolment Number
1	Shiwani Pathak	EN22CS301921
2	Shrishti Sahu	EN22CS301936
3	Shreyas	EN22CS301934
4	Sanskar Jain	EN22CS301868
5	Rajveer Singh Changal	EN23CS3T1009

Group Name: *Group 03D08*

Project Number: *GAI-38*

Industry Mentor Name: *Mr Yash*

University Mentor Name: *Prof. Divya Kumawat*

Academic Year: *2025-2026*

Table of Contents

1. Introduction

1.1. Scope of the Document
1.2. Intended Audience
1.3. System Overview

2. System Design

2.1. Application Design
2.2. Process Flow
2.3. Information Flow
2.4. Components Design
2.5. Key Design Considerations
2.6. API Catalogue

3. Data Design

3.1. Data Model
3.2. Data Access Mechanism
3.3. Data Retention Policies
3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements

7.1. Security Aspects
7.2. Performance Aspects

8. References

1. Introduction

The Construction Content Generator is a specialized Generative AI application designed to assist construction professionals in producing high-quality, industry-standard documentation. By leveraging advanced Large Language Model (LLM) capabilities through the Groq API and a persistent vector database (ChromaDB), the system transforms simple topic inputs into fully structured professional reports such as Site Reports, Safety Reports, Progress Reports, Inspection Reports, Daily Reports, and Material Reports.

1.1. Scope of the Document

This High-Level Design (HLD) document describes the overall architecture, system components, data flow, interfaces, and non-functional considerations for the Construction Content Generator project. It serves as a blueprint for understanding how the system is structured and how its components interact to deliver the intended functionality.

This document covers:

- Architecture and system design of the Flask-based backend
- Frontend interface design using HTML, CSS, and JavaScript
- Integration with Groq API (LLaMA 3.3-70B model) for LLM-based content generation
- Use of ChromaDB as a persistent vector database for semantic document retrieval
- PDF export functionality using ReportLab
- API endpoints, data flow, and component interactions

1.2. Intended Audience

This document is intended for the following stakeholders:

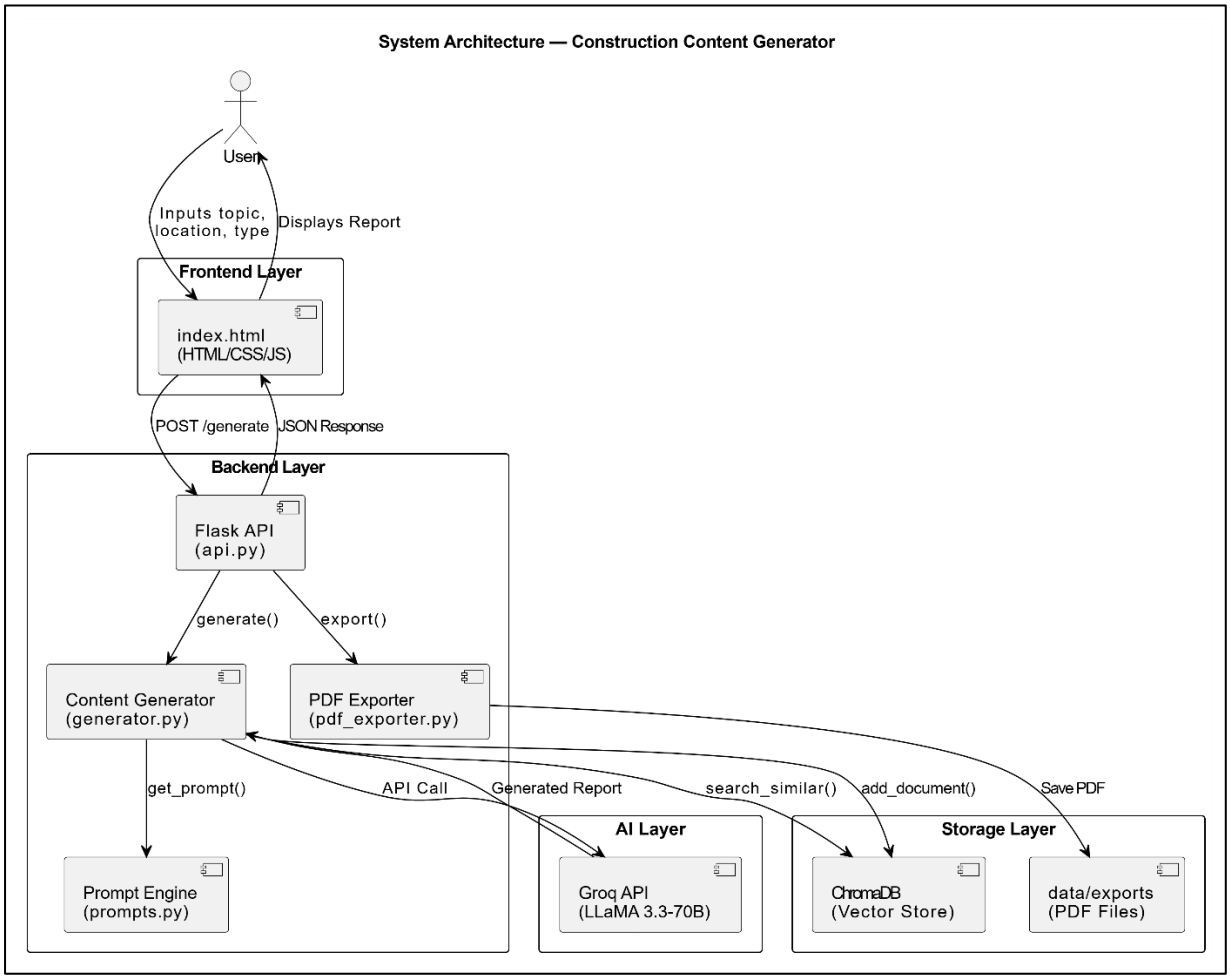
- University Mentors and Industry Mentors evaluating the project
- Developers who may maintain or extend the system in the future
- Technical reviewers assessing the architecture and design decisions
- Students presenting the project for the Medicaps University Datagami Skill Based Course

1.3. System Overview

The Construction Content Generator is a full-stack web application with the following high-level components:

- Frontend (index.html): A dark-themed, responsive single-page UI built with plain HTML, CSS, and JavaScript. Users select a report type, enter a topic and site location, and click Generate to receive a formatted professional report.
- Backend (api.py): A Flask REST API that handles report generation requests, coordinates between the LLM and vector store, generates PDF exports, and serves the frontend.
- Generator (app/generator.py): Core logic that retrieves similar past reports from ChromaDB, constructs prompt-engineered inputs, calls the Groq API, and stores the output back in the vector database.
- Vector Store (app/vector_store.py): ChromaDB PersistentClient storing report embeddings for semantic similarity search, enabling consistency across reports.
- PDF Exporter (app/pdf_exporter.py): ReportLab-based module that converts raw LLM output into structured, styled PDF documents saved to data/exports/.
- Prompt Templates (app/prompts.py): Six carefully engineered prompt templates ensuring professional tone, structure, and construction industry terminology for each report type.

2. System Design

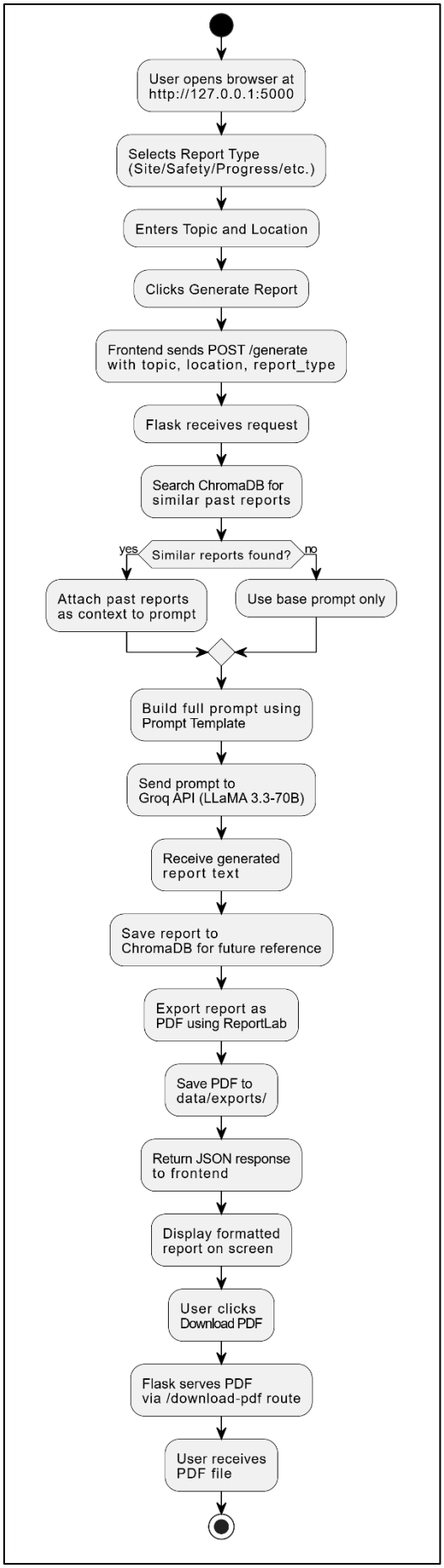


2.1. Application Design

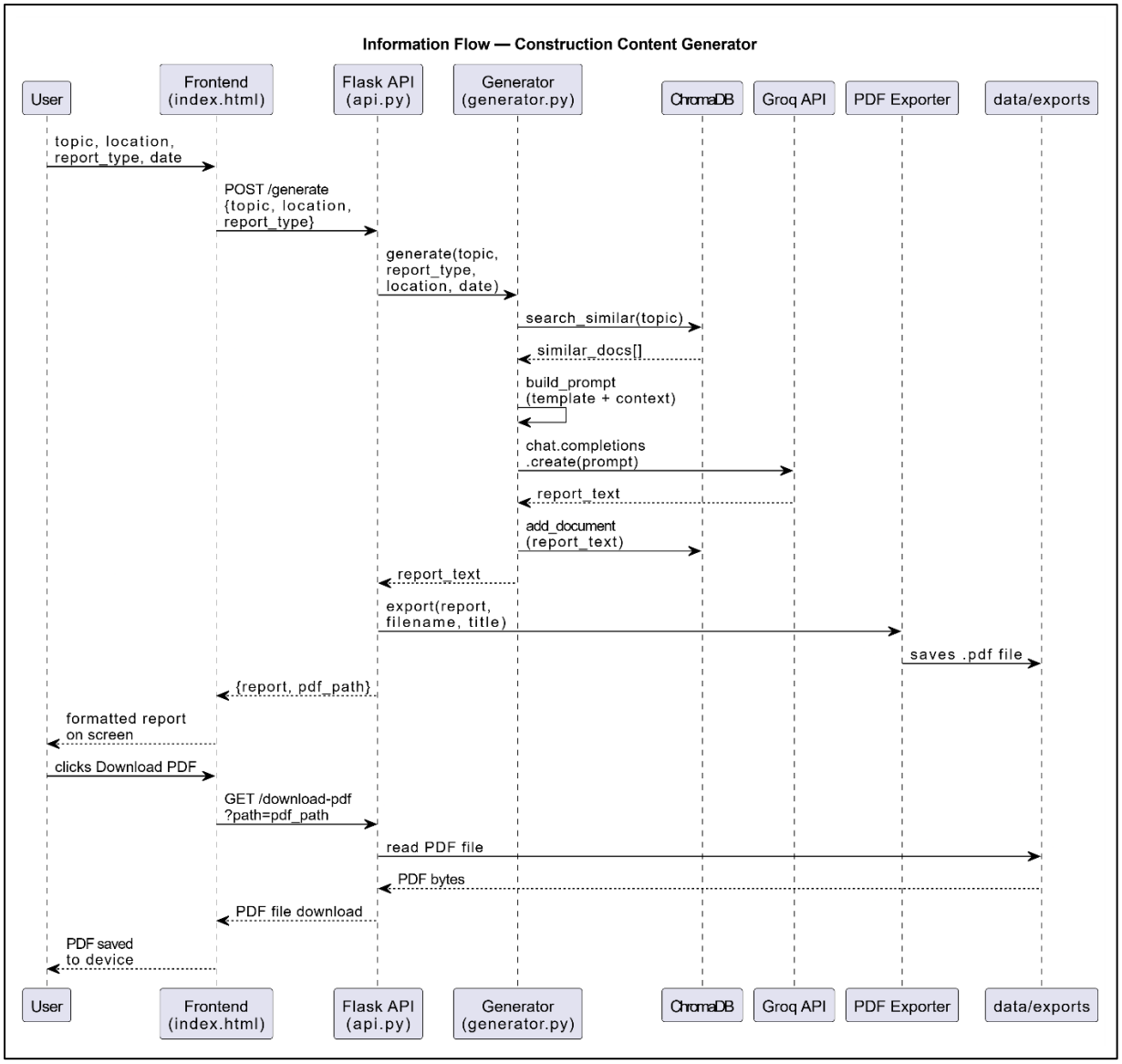
The application follows a simple client-server architecture:

Layer	Technology	Component	Purpose
Presentation	HTML / CSS / JS	index.html	User interface for report generation
Application	Python / Flask	api.py	REST API server, routing, orchestration
Business Logic	Python	generator.py, prompts.py	LLM calls, prompt engineering
Data	ChromaDB	vector_store.py	Semantic storage and retrieval
Export	ReportLab	pdf_exporter.py	PDF generation and formatting
LLM API	Groq (LLaMA 3.3-70B)	External API	AI text generation

2.2 Process Flow



2.3. Information Flow



Information flows through the system in the following direction:

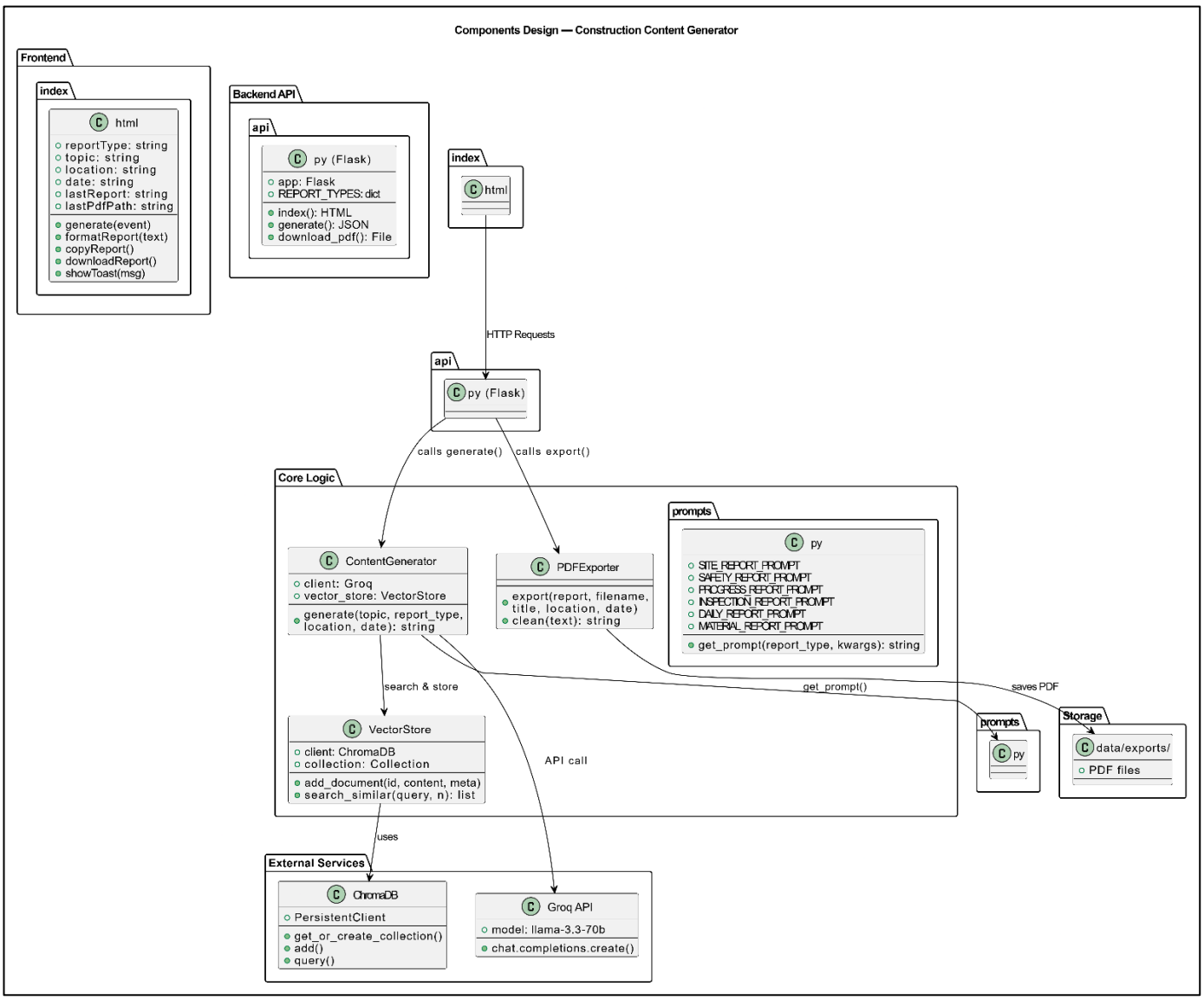
User Input (Browser) → HTTP POST /generate (Flask) → ContentGenerator → ChromaDB Query → Prompt Builder → Groq API → Report Text → ChromaDB Store + PDF Export → HTTP JSON Response → Browser Render

On PDF download, the flow is:

Browser → GET /download-pdf?path=... (Flask) → send_file() → PDF served to browser

2.4. Components Design

Component	Responsibility	Key Details
api.py	Flask server, routing, CORS, PDF download endpoint	Routes: GET /, POST /generate, GET /download-pdf
generator.py	Orchestrates vector search, prompt building, LLM call, storage	Model: llama-3.3-70b-versatile via Groq
prompts.py	Stores 6 prompt templates with structured section guidance	Site, Safety, Progress, Inspection, Daily, Material
vector_store.py	ChromaDB persistent client for semantic storage/retrieval	Path: ./data/chromadb/, n_results=3
pdf_exporter.py	Converts markdown report text to styled PDF	ReportLab, A4 size, orange headings
index.html	Single-page frontend with report type selector, inputs, output display	Fetch API, formatReport() markdown parser



2.5. Key Design Considerations

- Prompt Engineering: Each of the six report types has a dedicated, carefully crafted prompt template that instructs the LLM to use formal construction industry language, specific section headings, and no signature placeholders.
- Vector Augmented Generation: ChromaDB stores previously generated reports as embeddings. When a new report is requested, semantically similar past reports are retrieved and injected into the prompt as context, improving consistency over time.
- Separation of Concerns: The project is modular — generator, vector store, PDF exporter, and prompts are all separate Python modules, making maintenance straightforward.
- CORS Handling: Flask-CORS is used to allow the frontend (served by Flask itself at port 5000) to communicate with the API without cross-origin issues.
- Single Server Architecture: Both frontend (index.html) and API are served from the same Flask instance, eliminating CORS issues entirely when accessed via http://127.0.0.1:5000.

2.6. API Catalogue

Endpoint	Method	Request Body	Response	Description
GET /	GET	None	HTML page	Serves the frontend UI
POST /generate	POST	{ topic, location, report_type }	{ report, pdf }	Generates report and PDF
GET /download-pdf	GET	?path=<file_path>	PDF file	Downloads generated PDF

3. Data Design

3.1. Data Model

The system uses two forms of data storage:

A. ChromaDB Vector Store — stores generated reports as vector embeddings:

Field	Type	Description
id	String	Unique identifier: {report_type}_{topic[:30]}
document	String	Full text of the generated report
metadata.type	String	Report type (e.g., safety_report)
metadata.topic	String	Original topic entered by the user

B. File System Storage — PDF exports:

- Location: data/exports/
- Naming convention: {report_type}_{topic[:20]}_{date}.pdf
- Format: PDF generated by ReportLab

3.2. Data Access Mechanism

- ChromaDB is accessed via PersistentClient with the collection name construction_reports
- The default embedding function (all-MiniLM-L6-v2) is used for vectorizing report text
- Similarity search returns top 3 results (n_results=3) for context enrichment
- PDF files are accessed directly from the file system via Flask's send_file()

3.3. Data Retention Policies

- ChromaDB data persists indefinitely in the ./data/chromadb/ directory until manually deleted
- PDF exports persist indefinitely in ./data/exports/ until manually deleted
- No automatic data expiration or cleanup policy is implemented in the current version
- The .gitignore file excludes data/ and venv/ from version control

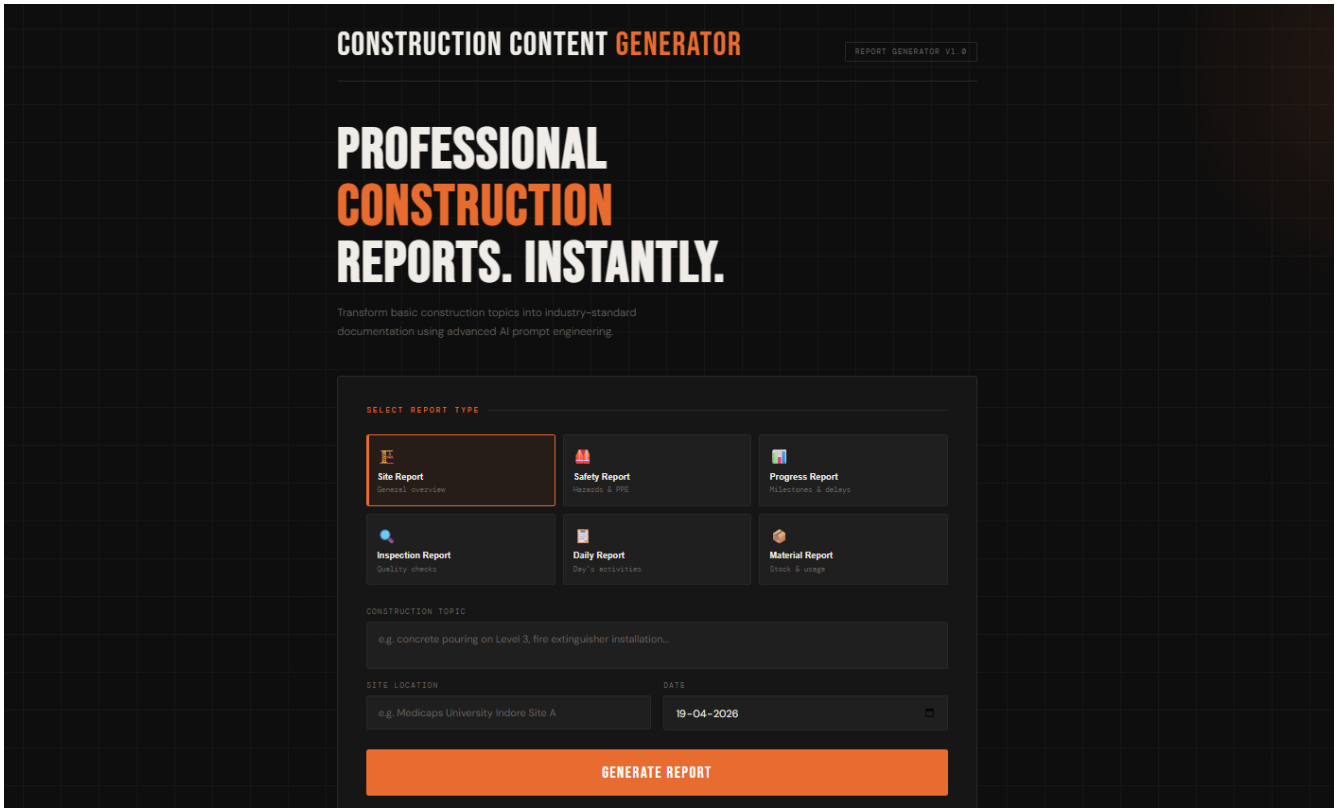
3.4. Data Migration

No formal data migration strategy is required for this project as it operates as a standalone local application. ChromaDB's PersistentClient stores data as local files which can be copied or moved if needed. Future migration considerations may include exporting ChromaDB collections to JSON for backup or transferring to a cloud-hosted vector database.

4. Interfaces

The system exposes and consumes the following interfaces:

- **User Interface:** A single-page web application (index.html) served at http://127.0.0.1:5000. The UI presents six report type buttons, a topic textarea, site location input, date picker, and a Generate Report button. Generated reports are displayed with formatted headings and bullet points
.



- **REST API Interface:** Flask exposes three HTTP endpoints (GET /, POST /generate, GET /download-pdf) as described in the API Catalogue. Communication is over HTTP with JSON payloads.
- **Groq LLM API Interface:** The system communicates with Groq's external API at https://api.groq.com using the groq Python SDK. Authentication is via GROQ_API_KEY stored in a .env file.
- **ChromaDB Interface:** Accessed programmatically via the chromadb Python library using PersistentClient. No external network connection required; runs entirely locally.

- **File System Interface:** PDF files are written to and read from the local ./data/exports/ directory using Python's file I/O and Flask's send_file() function.

5. State and Session Management

The Construction Content Generator is a stateless application by design:

- No user authentication or session management is implemented
- Each POST /generate request is fully self-contained with all required inputs (topic, location, report_type) in the request body
- The frontend maintains minimal in-memory state using JavaScript variables: selectedType (current report type), lastReport (last generated report text), and lastPdfPath (PDF file path for download)
- These frontend variables are reset on page refresh; no localStorage or sessionStorage is used
- The Flask backend is stateless between requests; all persistent state is in ChromaDB and the file system

6. Caching

The current version of the Construction Content Generator does not implement explicit caching. However, the following natural caching behavior exists:

- ChromaDB as Implicit Cache: Previously generated reports stored in ChromaDB serve as a form of semantic cache. When similar topics are requested, the vector store retrieves relevant past reports, reducing the need for the LLM to generate entirely from scratch.
- No HTTP Caching: No Cache-Control headers or server-side response caching is configured in Flask.
- No LLM Response Caching: Each API call to Groq generates a fresh response. Future versions could implement caching of identical topic+report_type combinations to reduce API usage.
- File System as Cache: Generated PDFs persist in data/exports/ and can be re-downloaded without regenerating if the file already exists.

7. Non-Functional Requirements

7.1. Security Aspects

- API Key Security: The Groq API key is stored in a .env file and loaded via python-dotenv. The .env file is excluded from version control via .gitignore.
- No Authentication: The application is designed for local use only and does not implement user authentication. It should not be deployed on a public server without adding authentication middleware.
- PDF Path Handling: The /download-pdf endpoint accepts a file path parameter. In a production environment, path traversal attacks must be mitigated by validating that the requested path is within the allowed exports directory.
- CORS: Flask-CORS is configured to allow all origins for development purposes. In production, this should be restricted to specific trusted origins.
- Input Validation: Basic validation is performed on the frontend (topic and location must be non-empty). Server-side input validation should be strengthened in future versions.

7.2. Performance Aspects

- LLM Latency: Report generation takes approximately 5–15 seconds depending on Groq API response time and report complexity. The frontend shows a loading animation during this period.
- ChromaDB Query Speed: Vector similarity search on a small local database (typically < 100 documents) is near-instantaneous (< 100ms).
- PDF Generation: ReportLab PDF generation for a typical report (500–1500 words) completes in under 1 second.
- Concurrent Requests: Flask's development server handles requests sequentially. For production use, a WSGI server (e.g., Gunicorn) should be used to support concurrent users.
- Model Selection: LLaMA 3.3-70B via Groq is chosen for its balance of output quality and speed. Groq's hardware acceleration provides significantly faster inference than standard cloud LLM providers.

8. References

- Groq API Documentation: https://console.groq.com/docs
- ChromaDB Documentation: https://docs.trychroma.com
- Flask Documentation: https://flask.palletsprojects.com
- ReportLab Documentation: https://www.reportlab.com/docs/reportlab-userguide.pdf
- LLaMA 3.3 Model Card: https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct
- Python-dotenv: https://pypi.org/project/python-dotenv/